30-Day Power Grid Project Blueprint

⚠️ Honest 30-Day Reality First

```
30 days is tight for this project.
We will build a COMPLETE working system.
But we must be focused and disciplined.

Daily commitment needed:
Minimum: 3 hours/day
Ideal:   4-5 hours/day

No skipping days.
Each day builds on previous.
Missing one day = two days behind.
```

📦 What We Will Have at Day 30

```
✅ Working IEEE 14-bus grid simulation
✅ 3 cooperative RL agents trained
✅ Safety layer implemented
✅ Renewable uncertainty handling
✅ 3 classical baselines compared
✅ Training curves and results plots
✅ Live dashboard
✅ Clean documented codebase
✅ README with results
✅ GitHub ready to publish
```

🗓️ Full 30-Day Blueprint

WEEK 1 — Days 1-7
Build The Grid Environment

Day 1 — Setup + Installation

```
Goal:
Everything installed and working

Tasks:
→ Create project folder structure
→ Install all dependencies
→ Verify each library works
→ Create base files

Install:
pip install pandapower
pip install gymnasium
pip install stable-baselines3
pip install torch
pip install cvxpy
pip install plotly
pip install dash
pip install numpy
pip install scipy
pip install matplotlib
pip install pettingzoo

Project Structure:
power_grid_marl/
│
├── env/
│   ├── grid_env.py        ← main environment
│   ├── grid_physics.py    ← swing equations
│   ├── grid_network.py    ← pandapower setup
│   └── renewable.py       ← solar/wind models
│
├── agents/
│   ├── dispatch_agent.py  ← agent 1
│   ├── frequency_agent.py ← agent 2
│   ├── voltage_agent.py   ← agent 3
│   └── safety_layer.py    ← safety filter
│
├── training/
│   ├── train_single.py    ← week 2
│   ├── train_marl.py      ← week 3
│   └── mappo.py           ← algorithm
│
├── baselines/
│   ├── droop_control.py   ← classical 1
│   ├── agc_control.py     ← classical 2
```

```
│   └── opf_baseline.py    ← classical 3
│
├── evaluation/
│   ├── metrics.py         ← performance
│   └── compare.py         ← baseline comparison
│
├── dashboard/
│   └── app.py             ← plotly dash
│
├── data/
│   ├── load_profiles/     ← NREL data
│   └── renewable_data/    ← solar/wind
│
├── results/
│   ├── plots/
│   └── models/
│
└── README.md
```

Deliverable:
✅ Project structure created
✅ All libraries installed
✅ Test import of each library works
```

---

### Day 2 — Pandapower Grid Setup
```
Goal:
IEEE 14-bus grid loaded and running

Tasks:
→ Load IEEE 14-bus in Pandapower
→ Run basic power flow
→ Understand bus/line structure
→ Print grid state

Key Code:
```
import pandapower as pp
import pandapower.networks as pn

net = pn.case14()  # IEEE 14-bus
pp.runpp(net)      # run power flow

print(net.bus)     # see all buses
print(net.gen)     # see generators
print(net.line)    # see lines
```

```
print(net.load)    # see loads

Deliverable:
✅ Grid loads without errors
✅ Power flow converges
✅ Understand what each bus/line is
✅ Can read voltage at every bus
```

Day 3 — Swing Equation Physics
```
Goal:
Grid dynamics simulated with ODEs

Tasks:
→ Implement swing equation
→ Simulate frequency response
→ Test disturbance (load step)
→ Plot frequency over time

Swing Equation:
M * d²δ/dt² + D * dδ/dt = Pm - Pe

Simplified frequency model:
df/dt = (Pm - Pe - D*Δf) / (2H)

Where:
f = frequency
Pm = mechanical power
Pe = electrical power
H = inertia constant
D = damping coefficient

Test:
→ Start at 50Hz
→ Add sudden load increase
→ Watch frequency drop
→ Verify physics makes sense

Deliverable:
✅ Swing equation implemented
✅ Frequency drops on load increase
✅ Frequency plot looks realistic
✅ Stable at 50Hz at rest
```

Day 4 — State Space Definition
```
Goal:
Define exactly what agents see

Tasks:
→ Define observation space
→ Define action space
→ Define reward function
→ Code state extraction

Observation Space (what agents see):
state = [
  # Frequency info
  frequency,              # Hz
  frequency_deviation,    # Hz from 50
  df_dt,                  # rate of change

  # Voltage info (14 buses)
  voltage_bus_1...14,     # per unit

  # Power info
  active_power_gen_1...5, # MW
  reactive_power_1...5,   # MVAR
  total_load,             # MW
  load_mismatch,          # MW imbalance

  # Line info
  line_loading_1...20,    # % of limit
]
Total: ~50 state variables

Action Space (what agents do):
Agent 1 (Dispatch):
  → Generator setpoints [MW]
  → 5 continuous values

Agent 2 (Frequency):
  → Fast reserve commands
  → 3 continuous values

Agent 3 (Voltage):
  → Reactive power injection
  → 4 continuous values

Reward:
```

```
reward = (
  -10 * abs(freq - 50)       # frequency
  -5  * voltage_violations    # voltage
  -1  * generation_cost       # cost
  -100* if_blackout           # catastrophic
  +1  * stability_bonus       # good behavior
)
```

Deliverable:
✅ State space coded
✅ Action space coded
✅ Reward function coded
✅ Can extract state from grid
```

---

### Day 5 — Gymnasium Environment
```
Goal:
Grid wrapped as proper Gym env

Tasks:
→ Build GridEnv class
→ Implement reset()
→ Implement step()
→ Implement render()
→ Test with random actions

class PowerGridEnv(gym.Env):

  def reset(self):
    → Load fresh IEEE 14-bus
    → Set initial conditions
    → Return initial state

  def step(self, action):
    → Apply action to grid
    → Run power flow
    → Integrate swing equation
    → Calculate reward
    → Check if done (blackout)
    → Return (state, reward, done, info)

  def render(self):
    → Print grid status
    → Show frequency, voltage
```
```

Test:
env = PowerGridEnv()
obs = env.reset()
for _ in range(100):
  action = env.action_space.sample()
  obs, reward, done, info = env.step(action)
  if done: break

Deliverable:
✅ Environment runs without errors
✅ Random actions work
✅ Reward changes with actions
✅ Done triggers on blackout
```

---

### Day 6 — Load Profiles + Disturbances
```
Goal:
Realistic varying demand in simulation

Tasks:
→ Download NREL load data
→ Implement daily load curve
→ Add random disturbances
→ Add fault injection

Load Profile:
→ Morning peak (8-10 AM)
→ Afternoon dip (2-3 PM)
→ Evening peak (6-8 PM)
→ Night low (12-5 AM)
→ Random noise on top

Disturbances:
→ Sudden load step (+/- 50MW)
→ Generator trip (one gen fails)
→ Line fault (one line disconnects)
→ Random load spikes

Deliverable:
✅ Load varies realistically
✅ Disturbances inject properly
✅ Grid responds to disturbances
✅ Environment is challenging
```
```

Day 7 — Environment Testing + Debug
```
Goal:
Environment is solid and bug-free

Tasks:
→ Run 1000 random episodes
→ Check for crashes
→ Verify physics makes sense
→ Check reward scaling
→ Fix all bugs found

Tests to run:
→ Does power flow always converge?
→ Does frequency stay physical?
→ Does reward punish blackouts?
→ Does state space have NaN values?
→ Does reset work cleanly?

Tune reward weights:
→ Print average reward
→ Adjust weights if needed
→ Make sure not too easy/hard

Deliverable:
✅ 1000 episodes run without crash
✅ Physics looks realistic
✅ Reward function makes sense
✅ Environment documented
✅ WEEK 1 COMPLETE
```

WEEK 2 — Days 8-14
Single Agent Baseline

Day 8 — PPO Single Agent Setup
```
Goal:
First RL agent training starts

Tasks:
→ Wrap env for single agent
```

→ Configure PPO from SB3
→ Start training
→ Monitor progress

```
from stable_baselines3 import PPO

model = PPO(
  "MlpPolicy",
  env,
  learning_rate=3e-4,
  n_steps=2048,
  batch_size=64,
  n_epochs=10,
  verbose=1
)

model.learn(total_timesteps=100_000)
```

Deliverable:
✅ Training starts without error
✅ Reward slowly increasing
✅ Agent not crashing grid
```

---

### Day 9 — Training + Monitoring
```
Goal:
Train single agent properly

Tasks:
→ Train for 500K timesteps
→ Log training metrics
→ Plot learning curves
→ Save best model

Monitor:
→ Episode reward mean
→ Frequency deviation
→ Voltage violations
→ Blackout count

Training will take:
~2-4 hours depending on CPU

Deliverable:
✅ 500K timesteps trained

✅ Learning curve plotted
✅ Model saved to results/models/
```

---

### Day 10 — Evaluate Single Agent
```
Goal:
See how well single agent works

Tasks:
→ Load trained model
→ Run 100 evaluation episodes
→ Compare vs no-control baseline
→ Plot performance metrics

Metrics to measure:
→ Average frequency deviation
→ % time voltage in safe range
→ Total generation cost
→ Number of blackouts
→ Recovery time after fault

No-control baseline:
→ Same environment
→ Zero action applied
→ See what happens naturally

Deliverable:
✅ Single agent clearly beats
   no-control baseline
✅ Numbers recorded for comparison
```

---

### Day 11 — Droop Control Baseline
```
Goal:
Implement classical droop controller

Tasks:
→ Code droop control math
→ Run on same environment
→ Record performance metrics
→ Compare to single agent

Droop Control (Pure Math):
ΔP = -R * Δf

Where:
ΔP = change in power output
R = droop coefficient (4%)
Δf = frequency deviation

Simple proportional response:
If frequency drops → increase generation
If frequency rises → decrease generation

No learning. Fixed formula.
This is what runs in real grids today.

Deliverable:
✅ Droop control coded
✅ Runs on environment
✅ Performance numbers recorded
```

---

### Day 12 — AGC Baseline
```
Goal:
Implement AGC (PI controller)

Tasks:
→ Code AGC math
→ Tune PI gains
→ Run on environment
→ Record metrics

AGC Formula:
ACE = Δf * B   (area control error)
u(t) = Kp*ACE + Ki*∫ACE dt

This is PI control on frequency.
Standard in every grid since 1950s.
Better than droop but still fixed.

Deliverable:
✅ AGC coded
✅ PI gains tuned
✅ Performance recorded
```
```

Day 13 — OPF Baseline
```
Goal:
Implement optimal power flow baseline

Tasks:
→ Use Pandapower built-in OPF
→ Run every N timesteps
→ Record performance
→ Note: OPF is slow (minutes)

import pandapower as pp
pp.runopp(net)  # optimal power flow

OPF gives theoretically optimal
dispatch for steady state.
Cannot handle dynamics.
Cannot run every second.

This is best classical method.
Our RL must eventually beat it
on combined metric.

Deliverable:
✅ OPF baseline running
✅ Performance recorded
✅ WEEK 2 COMPLETE
```

Day 14 — Week 2 Review
```
Goal:
Review all baselines so far

Compare everything so far:
→ No control
→ Droop
→ AGC
→ OPF
→ Single agent PPO

Make table of results.
Identify where single agent
is already winning.
```

Identify where it still loses.

This guides week 3 priorities.

Deliverable:
✅ Baseline comparison table
✅ Know what to improve in MARL
✅ Week 2 reviewed and documented
```

---

## WEEK 3 — Days 15-21
# Multi-Agent System

---

### Day 15 — Multi-Agent Environment
```
Goal:
Convert single env to multi-agent

Tasks:
→ Wrap with PettingZoo
→ Define per-agent observations
→ Define per-agent actions
→ Define per-agent rewards
→ Test with random actions

Key change:
Before: 1 agent sees everything
After: 3 agents each see
        their relevant state

Agent 1 sees: load, costs, generation
Agent 2 sees: frequency, df/dt, reserves
Agent 3 sees: voltages, reactive power

Shared reward + individual shaping

Deliverable:
✅ Multi-agent env working
✅ 3 agents step simultaneously
✅ Random actions run clean
```

---
```

Day 16 — MAPPO Implementation
```
Goal:
Implement MAPPO algorithm

Tasks:
→ Build centralized critic
→ Build 3 actor networks
→ Implement advantage estimation
→ Implement policy update

Why MAPPO not MADDPG:
→ MAPPO more stable
→ On-policy = more reliable
→ Simpler to implement
→ Better for cooperative tasks

Architecture:
Actor (per agent):
  Input: local observation
  Hidden: 128 → 128
  Output: action distribution

Critic (shared):
  Input: ALL agents observations
  Hidden: 256 → 256
  Output: value estimate

Deliverable:
✅ MAPPO coded from scratch
✅ Actor networks defined
✅ Critic network defined
✅ Forward pass works
```

Day 17 — Training Loop
```
Goal:
MAPPO training loop working

Tasks:
→ Implement rollout collection
→ Implement GAE (advantage)
→ Implement PPO update
→ Start training
```

Training Loop:
For each iteration:
1. Collect N timesteps
   from all 3 agents
2. Compute advantages
   using centralized critic
3. Update all 3 actors
   using PPO clipping
4. Update critic
5. Log metrics
6. Repeat

Start training:
1M timesteps
Will take 4-8 hours

Deliverable:
✅ Training loop running
✅ Loss decreasing
✅ Reward slowly increasing
✅ No crashes or NaN values
```

---

### Day 18 — Debug Training
```
Goal:
Fix training problems

Common problems:
→ Reward not increasing
  Fix: Adjust learning rate
       Check reward scaling

→ NaN values appear
  Fix: Gradient clipping
       Normalize observations

→ One agent dominates
  Fix: Individual reward shaping
       Balance action spaces

→ Divergence
  Fix: Reduce learning rate
       Increase batch size

This day is dedicated to

debugging whatever goes wrong.
This WILL happen.
Be prepared.

Deliverable:
✅ Training stable
✅ All 3 agents learning
✅ Reward curve going up
```

---

### Day 19 — Continue Training
```
Goal:
Let agents train fully

Tasks:
→ Train to 1M+ timesteps
→ Monitor every 50K steps
→ Save checkpoints
→ Adjust if needed

Checkpoints:
Save model every 100K steps
Compare performance over time
Keep best performing model

This day = mostly waiting
Use time to:
→ Write README
→ Plan evaluation
→ Prepare comparison metrics

Deliverable:
✅ 1M timesteps complete
✅ Multiple checkpoints saved
✅ Best model identified
```

---

### Day 20 — Safety Layer
```
Goal:
Add safety constraints to agents

Tasks:

→ Define safety bounds
→ Implement action projection
→ Add safety penalty to reward
→ Verify zero violations

Safety Bounds:
Frequency: 49.5 Hz - 50.5 Hz
Voltage: 0.95 pu - 1.05 pu
Line load: < 100% thermal limit
Generator: within min/max limits

Safety Layer Code:
```
def safe_action(action, grid_state):
  # Check each constraint
  if would_violate_freq(action):
    action = clip_to_safe(action)
  if would_violate_voltage(action):
    action = clip_to_safe(action)
  return action
```

Deliverable:
✅ Safety layer added
✅ Zero hard constraint violations
✅ Safety metrics tracked
```

---

### Day 21 — Renewable Uncertainty
```
Goal:
Add solar and wind to simulation

Tasks:
→ Add solar generation model
→ Add wind generation model
→ Retrain agents with renewables
→ Test robustness

Solar Model:
```
solar_power = peak * sin(π*hour/12)
            + noise * random.normal()
            + cloud_events
```

Wind Model:
```
wind_power = weibull_distribution()
           + sudden_stops
```

Effect:
→ Power supply unpredictable
→ Agents must be robust
→ Much harder than fixed load

Retrain:
→ Fine-tune existing model
→ 500K additional timesteps
→ With renewable noise

Deliverable:
✅ Renewables in simulation
✅ Agents handle fluctuations
✅ Performance still good
✅ WEEK 3 COMPLETE
```

---

## WEEK 4 — Days 22-30
# Evaluation + Dashboard + Polish

---

### Day 22 — Full Evaluation
```
Goal:
Rigorous comparison of all methods

Run every method on same scenarios:

Scenario 1: Normal operation
Scenario 2: Sudden load spike
Scenario 3: Generator failure
Scenario 4: Renewable fluctuation
Scenario 5: Multiple simultaneous faults

Methods compared:
1. No control
2. Droop control
3. AGC
4. OPF
5. Single agent PPO
6. Our MAPPO (3 agents)

Record for each:
→ Frequency deviation (mean + max)
→ Voltage violations (% time)

→ Generation cost ($/hr)
→ Blackout count
→ Recovery time (seconds)
→ Safety violations (target: 0)

Deliverable:
✅ Full results table
✅ Numbers for every method
✅ Our system wins on most metrics
```

---

### Day 23 — Results Plots
```
Goal:
Make publication-quality plots

Plots to create:

Plot 1: Training curve
  X: Training steps
  Y: Episode reward
  Shows: Agents improving over time

Plot 2: Frequency comparison
  X: Time (seconds)
  Y: Frequency (Hz)
  Lines: Each baseline + our method
  Shows: Our method stays closest to 50Hz

Plot 3: Voltage profile
  X: Bus number
  Y: Voltage (pu)
  Bars: Each method
  Shows: Our method keeps voltages safe

Plot 4: Cost comparison
  Bar chart
  Each method's generation cost
  Shows: We approach OPF optimality

Plot 5: Robustness under renewables
  X: Renewable penetration %
  Y: Performance metric
  Lines: Droop vs AGC vs Ours
  Shows: We degrade gracefully
```
```

Plot 6: Safety violations
  Bar chart
  Shows: We have zero violations

Tools: matplotlib + seaborn

Deliverable:
✅ 6 publication-quality plots
✅ Saved to results/plots/
✅ Tell clear story
```

---

### Day 24 — Dashboard Building
```
Goal:
Live interactive visualization

Build Plotly Dash dashboard:

Panel 1: Grid Topology Map
  → 14 buses shown as nodes
  → Lines shown as edges
  → Color = voltage level
  → Green = safe, Red = violation

Panel 2: Frequency Monitor
  → Real-time frequency plot
  → Red lines at 49.5/50.5 Hz
  → Updates every step

Panel 3: Voltage at Each Bus
  → Bar chart, 14 bars
  → Color coded safe/unsafe
  → Updates real-time

Panel 4: Agent Actions
  → What each agent is doing
  → Generator setpoints
  → Reactive power commands

Panel 5: Reward Tracker
  → Cumulative reward
  → Per-agent contribution

Panel 6: Event Log
  → Disturbances that occurred
```
```

→ Agent responses
→ Safety interventions

Deliverable:
✅ Dashboard runs locally
✅ Looks professional
✅ Updates in real-time
```

---

### Day 25 — Dashboard Polish
```
Goal:
Make dashboard impressive

Tasks:
→ Add play/pause control
→ Add scenario selector
→ Add speed control
→ Add comparison mode
→ Style with dark theme

Scenarios selectable:
→ Normal operation
→ Load spike test
→ Generator failure
→ Renewable storm
→ Worst case multi-fault

Comparison mode:
→ Show two methods side by side
→ Classic vs Our MARL
→ Visually shows improvement

Deliverable:
✅ Professional looking dashboard
✅ Multiple scenarios work
✅ Comparison mode working
```

---

### Day 26 — Code Cleanup
```
Goal:
Clean, documented, readable code

Tasks:
→ Add docstrings to every function
→ Add type hints
→ Remove debug print statements
→ Consistent naming conventions
→ Add comments to complex math
→ Create config file

Config file (config.yaml):
grid:
  bus_count: 14
  base_mva: 100

training:
  total_timesteps: 1000000
  learning_rate: 0.0003
  batch_size: 64

rewards:
  frequency_weight: 10.0
  voltage_weight: 5.0
  cost_weight: 1.0

safety:
  freq_min: 49.5
  freq_max: 50.5
  volt_min: 0.95
  volt_max: 1.05

Deliverable:
✅ All code documented
✅ Config file created
✅ No debug code left
✅ Consistent style
```

---

### Day 27 — README + Documentation
```
Goal:
World-class README

README Structure:

Multi-Agent Power Grid Control

What This Project Does

(Simple explanation)

The Problem We Solve
(Current grid limitations)

Our Innovation
(What gaps we fill)

Architecture
(System diagram)

Results
(Key numbers, plots)

Installation
pip install -r requirements.txt

Usage
python train.py
python dashboard/app.py

Project Structure
(File tree explained)

Baselines Comparison
(Results table)

Technical Details
(Algorithms, hyperparameters)

Future Work
(What could be improved)

References
(Papers we build on)

Deliverable:
✅ README is impressive
✅ Anyone can understand project
✅ Anyone can run project
✅ Results clearly shown
```

---

### Day 28 — Testing + Bug Fixes

```

Goal:
Everything works perfectly

Tests:
→ Fresh install on clean environment
→ Training runs end to end
→ All baselines run
→ Dashboard launches
→ All plots generate
→ Config changes work

Fix every bug found today.
This is critical before GitHub publish.

Deliverable:
✅ Zero critical bugs
✅ Runs from fresh install
✅ All features work
```

---

Day -29
Goal:
Professional GitHub repository

Tasks:
→ Initialize git repo
→ Write .gitignore
→ Create requirements.txt
→ Add license (MIT)
→ Create demo GIF for README
→ Push to GitHub

requirements.txt:
pandapower==2.13.1
gymnasium==0.29.0
stable-baselines3==2.2.1
torch==2.1.0
cvxpy==1.4.1
plotly==5.18.0
dash==2.14.0
numpy==1.26.0
scipy==1.11.4
matplotlib==3.8.0
pettingzoo==1.24.1

Demo GIF:
```

→ Record dashboard running
→ Show agent stabilizing grid
→ Show frequency recovering
→ Put in README

Deliverable:
✅ GitHub repo live
✅ Professional appearance
✅ Demo GIF in README
✅ Requirements documented

Day 30

Goal:
Complete project reviewed

Final checklist:
✅ Grid simulation working
✅ 3 agents trained
✅ Safety layer active
✅ Renewables handled
✅ 5 baselines compared
✅ 6 result plots generated
✅ Dashboard running
✅ Code clean + documented
✅ README complete
✅ GitHub published

Write future work section:
→ Scale to IEEE 118-bus
→ Add GNN agent architecture
→ Formal safety proofs
→ Real SCADA data integration
→ Multi-area grid coordination

Deliverable:
✅ Complete project
✅ GitHub published
✅ Portfolio ready
✅ Paper ready to write